Oasis CLI 설정 시스템 상세 분석

개요

Oasis CLI는 TOML 기반의 중앙화된 설정 관리 시스템을 사용하여 네트워크, 지갑, 주소록 등의 구성 정보를 관리합니다. 이 문서는 설정 시스템의 구조, 로딩 과정, 마이그레이션 메커니즘, 그리고 검증 과정을 상세히 분석합니다.

설정 파일 구조

1. 설정 파일 위치 및 기본 정보

설정 파일 경로: `$XDG_CONFIG_HOME/oasis/cli.toml`

- Linux/macOS: `~/.config/oasis/cli.toml`
- Windows: `%USERPROFILE%\AppData\Local\oasis\cli.toml`

파일 형식: TOML (Tom's Obvious, Minimal Language)

- 사람이 읽기 쉬운 구조
- 계층적 데이터 표현 지원
- 타입 안전성 보장

2. 핵심 설정 구조

```
type Config struct {
Networks config.Networks // 네트워크 설정
Wallet Wallet // 지갑 설정
```

```
AddressBook AddressBook // 주소록 설정
LastMigration  int  // 마이그레이션 버전
}
```

2.1 네트워크 설정 (Networks)

네트워크 설정은 다음과 같은 구조를 가집니다:

```
[networks]
default = "mainnet"

[networks.all.mainnet]
chain_context = "b11b369e0da5bb230b220127f5e7b242d385ef8c6f54906243f30af63c81553
rpc = "grpc.oasis.io:443"

[networks.all.mainnet.paratimes]
default = "sapphire"

[networks.all.mainnet.paratimes.all.sapphire]
id = "0000000000000000000000000000000000000000000000f80306c9858e7279"
description = "Sapphire"
consensus_denomination = "ROSE"

[networks.all.mainnet.paratimes.all.sapphire.denominations.ROSE]
```

```
symbol = "ROSE"

decimals = 18
```

주요 구성 요소:

- `chain_context` : 블록체인 네트워크 식별자
- `rpc` : gRPC 엔드포인트 주소
- `paratimes` : 지원되는 ParaTime 목록과 설정
- `denominations` : 토큰 단위 및 소수점 정보

2.2 지갑 설정 (Wallet)

```
[wallets]

default = "my-wallet"


[wallets.my-wallet]

description = "My primary wallet"

kind = "file"

address = "oasis1qp0l8r2s3076n4xatn8xjlkr74dh9hkdvqgqjqjx"

algorithm = "ed25519-adr8"

number = 0
```

주요 구성 요소:

- `kind` : 지갑 백엔드 유형 (file, ledger)
- `algorithm` : 암호화 알고리즘 (ed25519-adr8, secp256k1-bip44 등)
- `address` : 계정 주소

- `number` : 계정 번호 (HD 지갑의 경우)

2.3 주소록 설정 (AddressBook)

```
[address_book.alice]
description = "Alice's account"
address = "oasis1qp0l8r2s3076n4xatn8xjlkr74dh9hkdvqgqjqjx"
eth_address = "0x1234567890123456789012345678901234567890"
```

설정 로딩 과정

1. 초기화 단계 (`initConfig()`)

설정 로딩은 `cmd/root.go` 의 `initConfig()` 함수에서 시작됩니다:

```go
func initConfig() {
v := viper.New()
if cfgFile != "" {
v.SetConfigFile(cfgFile)
} else {
const configFilename = "cli.toml"
configDir := config.DefaultDirectory()
configPath := filepath.Join(configDir, configFilename)
v.AddConfigPath(configDir)
v.SetConfigType("toml")
```

```
v.SetConfigName(configFilename)

}

}
```

2. 설정 파일 검색 및 생성

검색 순서:

1. `--config` 플래그로 지정된 파일
2. 기본 설정 디렉토리의 `cli.toml`

파일이 없는 경우:

```
if _, err := os.Stat(configPath); os.IsNotExist(err) {

if err := os.MkdirAll(configDir, 0o700); err != nil {

cobra.CheckErr(err)

}

// 기본 설정으로 초기 설정 파일 생성

config.ResetDefaults()

_ = config.Save(v)

}
```

3. 설정 로딩 및 검증

```
// 전역 설정 로드 및 검증

err := config.Load(v)
```

```
cobra.CheckErr(err)



changes, err := config.Global().Migrate()

cobra.CheckErr(err)



if  changes {

err = config.Global().Save()

cobra.CheckErr(err)

}



err = config.Global().Validate()

cobra.CheckErr(err)
```

TOML 기반 설정 관리 시스템

1. Viper 라이브러리 활용

Oasis CLI는 Viper 라이브러리를 사용하여 TOML 설정을 관리합니다:

주요 특징:

- 다양한 설정 형식 지원 (TOML, JSON, YAML 등)
- 환경 변수 오버라이드 지원
- 실시간 설정 변경 감지
- 계층적 설정 구조 지원

2. 구조체 매핑 (mapstructure)

설정 데이터는 mapstructure 태그를 사용하여 Go 구조체에 매핑됩니다:

```
type Config struct {
Networks config.Networks `mapstructure:"networks"`
Wallet Wallet `mapstructure:"wallets"`
AddressBook AddressBook `mapstructure:"address_book"`
LastMigration int `mapstructure:"last_migration"`
}
```

특수 태그:

- `remain` : 나머지 필드를 맵으로 처리
- `omitempty` : 빈 값일 때 생략

3. 설정 인코딩 및 저장

설정 저장 시 구조체를 맵으로 변환하는 `encode()` 함수를 사용:

```
func encode(in interface{}) (interface{}, error) {
v := reflect.ValueOf(in)
if v.Kind() == reflect.Ptr {
v = v.Elem()
}
switch v.Kind() {
case reflect.Struct:
result := make(map[string]interface{})
// 구조체 필드를 맵으로 변환
```

```
// mapstructure 태그 처리

// remain 속성 처리

return result, nil

case reflect.Map:

// 맵을 map[string]interface{}로 변환

return result, nil

default:

return v.Interface(), nil

}

}
```

설정 마이그레이션 시스템

1. 마이그레이션 버전 관리

```
const latestMigrationVersion = 1


var migrations = map[int]migrationFunc{

0: migrateFromV0,

}
```

2. 마이그레이션 실행 과정

```
func (cfg *Config) migrateVersions() (bool, error) {
var changes bool
for v := cfg.LastMigration; v < latestMigrationVersion; v++ {
fn, ok := migrations[v]
if !ok {
return false, fmt.Errorf("missing migration from version %d", v)
}
ch, err := fn(cfg)
if err != nil {
return false, fmt.Errorf("failed to apply migration from version %d: %w", v, err
}
changes = changes || ch
cfg.LastMigration = v + 1
}
return changes, nil
}
```

3. 네트워크 설정 마이그레이션

체인 컨텍스트 업데이트:

```
func (cfg *Config) migrateNetworks() bool {
var changes bool
```

```
for name, net := range cfg.Networks.All {
defaultCfg, knownDefault := Default.Networks.All[name]
oldNetwork, knownOld := OldNetworks[name]
// 이전 체인 컨텍스트를 새 것으로 교체
for _, oldChainContext := range oldNetwork.ChainContexts {
if net.ChainContext == oldChainContext {
net.ChainContext = defaultCfg.ChainContext
changes = true
break
}
}
}
return changes
}
```

ParaTime 설정 마이그레이션:

- 새로운 ParaTime 추가
- 기존 ParaTime 설정 업데이트
- 합의 계층 denomination 변경

4. 지갑 설정 마이그레이션

각 지갑 백엔드는 자체 마이그레이션 로직을 구현:

```
func (w *Wallet) Migrate() (bool, error) {
```

```
var changes bool
for _, acc := range w.All {
af, err := acc.LoadFactory()
if err != nil {
return false, err
}
changes = af.Migrate(acc.Config) || changes
}
return changes, nil
}
```

설정 검증 과정

1. 계층적 검증 구조

```
func (cfg *Config) Validate() error {
if err := cfg.Networks.Validate(); err != nil {
return fmt.Errorf("failed to validate network configuration: %w", err)
}
if err := cfg.Wallet.Validate(); err != nil {
return fmt.Errorf("failed to validate wallet configuration: %w", err)
}
return nil
```

```
}
```

2. 네트워크 설정 검증

- 체인 컨텍스트 형식 검증
- RPC 엔드포인트 유효성 확인
- ParaTime ID 형식 검증
- Denomination 설정 검증

3. 지갑 설정 검증

```
func (w *Wallet) Validate() error {
// 기본 계정 존재 확인
if _, exists := w.All[w.Default]; w.Default != "" && !exists {
return fmt.Errorf("default account '%s' does not exist in the wallet", w.Defaul
}
// 모든 계정 유효성 검사
for name, acc := range w.All {
if err := config.ValidateIdentifier(name); err != nil {
return fmt.Errorf("malformed account name '%s': %w", name, err)
}
if err := acc.Validate(); err != nil {
return fmt.Errorf("account '%s': %w", name, err)
}
}
```

```
return nil
}
```

4. 계정 설정 검증

```
func (a *Account) Validate() error {
// 지원되는 지갑 종류 확인
if _, err := wallet.Load(a.Kind); err != nil {
return fmt.Errorf("kind '%s' is not supported", a.Kind)
}
// 주소 형식 검증
var address types.Address
if err := address.UnmarshalText([]byte(a.Address)); err != nil {
return fmt.Errorf("malformed address '%s': %w", a.Address, err)
}
// 알고리즘 필드 존재 확인
if _, ok := a.Config["algorithm"]; !ok {
return fmt.Errorf("algorithm field not defined")
}
return nil
}
```

5. 주소록 설정 검증

```
func (ab *AddressBook) Validate() error {
for name, a := range ab.All {
if err := config.ValidateIdentifier(name); err != nil {
return fmt.Errorf("malformed address name '%s': %w", name, err)
}
if err := a.Validate(); err != nil {
return fmt.Errorf("address '%s': %w", name, err)
}
}
return nil
}
```

설정 시스템의 고급 기능

1. 동적 설정 업데이트

설정 변경 시 자동으로 파일에 저장:

```
func (cfg *Config) Save() error {
if err := cfg.Validate(); err != nil {
return err
}
encCfg, err := encode(cfg)
if err != nil {
```

```
return err
}
rawCfg := encCfg.(map[string]interface{})
// 설정 초기화 후 새 맵으로 병합
var buf bytes.Buffer
_ = cfg.viper.ReadConfig(&buf)
if err = cfg.viper.MergeConfigMap(rawCfg); err != nil {
return err
}
return cfg.viper.WriteConfig()
}
```

2. 기본값 관리

```
var Default = Config{
Networks: config.DefaultNetworks,
LastMigration: latestMigrationVersion,
}
```

3. 이전 버전 호환성

```
var OldNetworks = map[string]*OldNetwork{
"mainnet": {
```

```
ChainContexts: []string{
"b11b369e0da5bb230b220127f5e7b242d385ef8c6f54906243f30af63c815535",
},
RPCs: []string{
"grpc.oasis.dev:443",
},
},
}
```

설정 시스템의 보안 고려사항

1. 파일 권한 관리

- 설정 디렉토리: `0o700` (소유자만 읽기/쓰기/실행)
- 지갑 파일: 암호화된 JSON 형태로 저장
- 민감한 정보는 메모리에서 즉시 삭제

2. 설정 검증

- 모든 입력값에 대한 엄격한 검증
- 타입 안전성 보장
- 악의적인 설정 값 차단

3. 마이그레이션 안전성

- 백업 없이는 마이그레이션 실행 안 함

- 실패 시 롤백 메커니즘
- 버전 호환성 검사

성능 최적화

1. 지연 로딩

- 필요한 시점에만 설정 로드
- 메모리 사용량 최적화
- 빠른 CLI 시작 시간

2. 캐싱

- 전역 설정 객체 캐싱
- 반복적인 파일 I/O 방지
- 설정 변경 시에만 재로드

3. 효율적인 직렬화

- 구조체 태그를 활용한 최적화
- 불필요한 필드 제외
- 압축된 TOML 출력

확장성 및 유지보수성

1. 모듈화된 설계

- 각 설정 영역별 독립적인 구조체
- 인터페이스 기반 확장

- 플러그인 아키텍처 지원

2. 버전 관리

- 명확한 마이그레이션 경로
- 하위 호환성 보장
- 점진적 업그레이드 지원

3. 테스트 가능성

- 모든 설정 로직에 대한 단위 테스트
- 마이그레이션 시나리오 테스트
- 검증 로직 테스트

이 설정 시스템은 Oasis CLI의 핵심 인프라로서, 사용자 경험과 시스템 안정성을 모두 고려한 견고한 설계를 보여줍니다.